

ALGOLENS

AI Driven Code Performance Analyzer

ANALYSIS REPORT

Overall Score	Quality Score	Performance Score
98/100	100/100	95/100

1. Code Metrics

Metric	Value
Lines of Code	5
Functions	1
Loops	1
Conditions	0
Variables	1

2. Performance Analysis

Metric	Result
Time Complexity	O(n)
Space Complexity	O(1)
Maximum Loop Depth	1
Performance Score	95/100

3. Code Quality

Quality Score: 100/100

- No major code quality issues detected.

4. Performance Issues

- No major performance issues detected.

5. AI Code Summary

1. What the Code Does

Calculates and returns the sum of all non-negative integers strictly less than `n` (i.e., from `0` up to `n - 1`).

2. Main Purpose of the Code

To demonstrate basic iteration using a `for` loop and an accumulator variable to sum a sequence of numbers up to a specified limit.

3. How the Code Works

1. **Initializes the Accumulator:** Sets a variable `total` to `0`.
2. **Loops Through Numbers:** Uses `range(n)` to generate numbers starting from `0` up to `n - 1`.
3. **Accumulates Values:** Adds the current loop value (`i`) to `total` during each iteration.
4. **Returns Result:** Returns the final `total` after the loop finishes.

4. Important Observations

- * **Excludes `n`:** Because Python's `range(n)` is exclusive of the upper bound, calling `calculate\_sum(5)` sums `0 + 1 + 2 + 3 + 4` (resulting in `10`), not including `5`.
- * **Performance ($O(n)$):** The code runs in linear time $O(n)$ because it iterates n times.
- * **Mathematical Optimization:** This could be calculated in constant time $O(1)$ using the formula $(n * (n - 1)) // 2$ or built-in functions like `sum(range(n))`.
- * **Edge Cases:** If `n` is `0` or a negative integer, the loop does not execute, and the function returns `0`.

6. AI Improvement Suggestions

Problems Detected

- **Suboptimal Time Complexity:** The code uses a `for` loop to compute the sum, taking $O(n)$ time. As n grows large, performance drops significantly.
- **Unnecessary Iteration:** The sum of consecutive integers from 0 to $n-1$ can be calculated instantly using a standard mathematical formula rather than iterating step-by-step.

Improvement Suggestions

1. **Use the Arithmetic Series Formula:** Replace the loop with the mathematical formula $n \times (n - 1) / 2$ to achieve $O(1)$ time complexity.
2. **Use Integer Division (`//`):** Ensure the result remains an integer type rather than converting to a float.
3. **Add Type Hints and Docstring:** Enhance readability and maintainability by adding type hints (`n: int -> int`) and a brief docstring.

Optimization Explanation

1. Mathematical Approach (Best Optimization)

Instead of adding numbers one by one in a loop, you can calculate the total instantly using the formula for the sum of an arithmetic sequence:

$$\text{Sum} = \frac{n \times (n - 1)}{2}$$

- * **Time Complexity:** $O(1)$ — Performs a fixed number of operations regardless of how large n is.
- * **Space Complexity:** $O(1)$ — Uses a constant amount of memory.

```
```python
def calculate_sum(n: int) -> int:
 """Calculates the sum of integers from 0 up to n-1."""
 if n <= 0:
 return 0
 return n * (n - 1) // 2
```
```

2. Python Built-in Approach (Alternative)

If you prefer readable, idiomatic Python without writing manual math logic, Python's built-in `sum()` function is implemented in C and runs much faster than an explicit `for` loop, though it still has $O(n)$ time complexity:

```
```python
def calculate_sum(n: int) -> int:
 return sum(range(max(0, n)))
```
```

Performance Comparison

| Approach | Time Complexity | Space Complexity | Performance (n = 10,000,000) |
|----------------------------|-----------------|------------------|----------------------------------|
| Original Loop | $O(n)$ | $O(1)$ | Slow (~0.5 seconds) |
| <code>sum(range(n))</code> | $O(n)$ | $O(1)$ | Faster (~0.15 seconds) |
| Math Formula | $O(1)$ | $O(1)$ | Instantaneous (<0.00001 seconds) |

7. AI Improved Code

```
def calculate_sum(n):
    return n * (n - 1) // 2 if n > 0 else 0
```

Generated by Algolens — AI Driven Code Performance Analyzer